Beyond Hallucination Detection: Measuring Consequential Uncertainty in LLM-Generated Code

Michael Jaquith
Independent Researcher
michael.j.jaquith@gmail.com

April 2026

Abstract

Large language models generate code that appears uniformly confident. Developers receive no signal for which decisions warrant careful review — yet the failure modes of code generation are not random: they cluster around API version decisions, library selection, and architectural compatibility choices that are systematically expensive when wrong.

Existing uncertainty quantification methods address a structurally different problem. Multi-sample consistency methods (semantic entropy, UQLM) detect uncertainty by sampling a prompt repeatedly and measuring variance across outputs. For code generation, this approach fails: models have consistent training-data priors and generate the same API choice across all samples, regardless of whether that choice is correct. We term this the *intra- vs. inter-generation distinction*: code generation uncertainty exists at generation time, in the token-level probability distribution, and is gone from the committed output. It cannot be observed by repeated sampling.

ε is a *decision-risk prioritization signal*: it identifies where the model was uncertain at generation time, not where the code is incorrect. We introduce ε (epsilon), a token-level uncertainty primitive that operates within a single generation using log-probabilities. We identify three categories of token-level uncertainty in generated code — *cosmetic* (naming), *consequential* (API and library decisions), and *implementation-approach* (algorithm style) — and show that only consequential uncertainty correlates with real risk. We filter cosmetic uncertainty using AST-based declaration classification, isolating the signal that matters.

On a 30-prompt benchmark spanning three OpenAI models, simple algorithms, and version-split API calls, ε achieves an 85% detection rate on GPT-4o; 70% on two additional models. UQLM, the closest published baseline, achieves 5% detection (1 of 20 high-uncertainty prompts) compared to 85% for ε — empirically confirming that multi-sample methods are ineffective on this problem class, consistent with a structural argument we develop in Section 5. The ε system adds zero latency (log-probabilities arrive with the generation) and provides token-, line-, and function-level localization that multi-sample methods cannot produce.

1 Introduction

When a large language model generates a factual answer, uncertainty manifests as *inter-generational variance*: ask the same question ten times and you may observe different answers. Semantic entropy [10] and related methods detect this variance by clustering multiple responses and measuring their semantic spread. The underlying assumption — that uncertainty produces observable inconsistency across outputs — holds for factual generation.

Code generation breaks this assumption. When GPT-4o is asked to integrate the Stripe payment API, it chooses between the deprecated `Charge.create` method and the current `PaymentIntent`

workflow. This is a consequential decision. But the model has a strong training-data prior for one choice: it generates the same API call across all samples, consistently and confidently. The uncertainty that exists at generation time — visible in the token-level log-probability distribution as a near-50/50 split between the two API paths — is committed and gone from the output. Repeated sampling detects nothing.

We call this the *intra- vs. inter-generation distinction*. Code generation uncertainty is intra-generational: it lives inside a single forward pass, token by token, and must be measured there. This observation is not merely conceptual — we demonstrate it empirically in Section 5: UQLM [5] (a multi-sample semantic-entropy tool), the closest published tool for LLM uncertainty quantification, achieves 5% detection on a benchmark of API-version-split code generation prompts where our system achieves 85%.

The practical consequence is that AI code generation tools — Copilot, Cursor, Claude, ChatGPT — produce output that looks uniformly confident. No signal distinguishes a generation where the model was certain from one where it was choosing between a deprecated API that will crash in production and its modern replacement. The consequences are not evenly distributed. A sorting function with an unusual variable name is harmless. A payment processing function that calls a removed API method is a production incident. A FastAPI endpoint that uses `requests.get()` inside an `async def` will pass all unit tests and silently block the event loop under real load. The developer who reviews all generated code equally is reviewing the wrong things. The intra-generational nature of this uncertainty means the only viable measurement point is inside the generation itself — at the log-probability distribution over token alternatives at each position.

We introduce ε (epsilon), a token-level uncertainty primitive computed from the log-probability distribution returned by the model during a single generation. For each token position, ε_t is the normalized Shannon entropy over the top-5 alternative tokens. The file- or function-level score is the maximum ε_t across consequential code tokens. Three design choices distinguish ε from naive entropy measurement:

1. **AST-based cosmetic filtering.** The model's uncertainty about whether to name a parameter `n` or `num` is real entropy but zero risk. We parse the generated code with Python's `ast` module and exclude tokens at declaration positions (function names, parameter names, local variable names) from aggregation.
2. **Max aggregation, not compound.** The compound formula $1 - \prod(1 - \varepsilon_i)$ converges to 1.0 for any non-trivial response length, making it useless as a threshold signal. Max asks the right question: *how uncertain was the model at its single worst consequential decision?*
3. **Zero added latency.** Log-probabilities are returned by the API alongside the generation. The ε computation runs on the already- received data; no additional API call is required.

This paper makes four contributions:

1. **Intra- vs. inter-generation distinction** (§5). A conceptual framing that explains why multi-sample uncertainty quantification methods are structurally ineffective for code generation API decisions, with empirical confirmation: 5% vs. 85% detection rate on the same benchmark (UQLM vs. ε, GPT-4o).
2. **The ε primitive** (§3). A single-generation, token-level, AST-filtered uncertainty metric for code, with a three-tier threshold protocol and adaptive ensemble threshold.
3. **Failure taxonomy** (§4). Four classes of LLM code generation failure — deprecated API, confident wrongness, syntax split, and compatibility mismatch — each demonstrated with a calibrated scenario and live API results.

4. **Uncertainty taxonomy** (§3). Three types of token-level uncertainty in generated code: cosmetic, consequential, and implementation-approach. An empirical benchmark characterizes each type and demonstrates AST filtering's effect on false positive rate.

The implementation, benchmark, and all scenario scripts are available at https://github.com/michaeljaquith/epsilon-code.

2 Related Work

2.1 LLM Uncertainty Quantification

Semantic entropy. Kuhn et al. [10] introduce semantic entropy as a method for detecting hallucinations in free-form text generation. Multiple responses to the same prompt are generated and clustered by semantic equivalence (via natural language inference); high entropy across clusters indicates uncertainty. The method is principled and effective for factual question answering, where model uncertainty produces cross-generation variance. As we show in Section 5, this variance is not present for code generation API decisions: models generate the same (potentially wrong) API call across all samples.

UQLM. The UQLM library [5] provides open-source implementations of multiple uncertainty scoring methods for LLM outputs, including semantic negentropy, semantic clustering, and conformal prediction wrappers. It is the closest published tool to our work and serves as our primary baseline. Our direct comparison (Section 5) shows 5% detection rate (1 of 20 high-uncertainty prompts) compared to 85% for ε.

Conformal prediction for LLMs. Quach et al. [11] and Angelopoulos et al. [1] apply conformal prediction to construct prediction sets with coverage guarantees over LLM outputs. The approach is statistically rigorous but produces set-valued predictions over outputs rather than within-generation token-level signals. It also requires a calibration dataset and does not provide the localization (which token, which line, which function) that makes ε actionable for code review.

Token-level calibration. Kadavath et al. [9] show that LLMs are reasonably well-calibrated at the token level: stated confidence correlates with accuracy. Varshney et al. [12] use token-level log-probabilities for selective generation, abstaining when confidence is low. Our work extends this line by identifying *which* tokens carry meaningful uncertainty in code — specifically, by filtering out cosmetic naming decisions that dominate naive entropy measurements. Uncertainty propagation through multi-stage ML pipelines is well-studied in adjacent domains [8]; the ε primitive is designed to support analogous propagation in code generation pipelines, deferred to future work.

Code generation uncertainty and self-consistency. Concurrent approaches to code generation quality estimation include test-based ranking (CodeT [3] generates tests and ranks candidate solutions by pass rate) and repeated-sampling self-consistency (AlphaCode-style approaches use pass@k as an implicit uncertainty proxy). These methods are inter-generational by construction: they require multiple independent samples and aggregate across them. They are therefore subject to the same structural limitation as UQLM for the problem we study — a model that consistently generates the same API version across samples will produce consistent test outcomes regardless of whether the API version is correct. Our approach is complementary: it operates within a single generation and is zero-overhead.

2.2 Code Generation Quality

Recent work on evaluating LLM code generation quality focuses primarily on functional correctness via execution-based benchmarks (HumanEval [4], MBPP [2], SWE-bench [7]). These benchmarks measure whether generated code passes test cases, not whether the model was confident in its decisions. A model that passes all tests with low ε is reliable; a model that passes all tests with high ε on specific API decisions may be one dependency update away from failure.

To our knowledge, no prior work treats within-generation token-level uncertainty as a first-class signal for code generation risk. The closest work is in automated program repair, where uncertainty has been used to rank repair candidates [13], but this operates post-generation and does not address the intra-generation measurement problem we identify.

3 The ε Primitive

3.1 Token-Level Uncertainty

Modern LLM APIs return, alongside each generated token, a log-probability distribution over the top-k alternative tokens at that position. For a token at position t with top-k alternatives $\{(v_1, p_1), \ldots, (v_k, p_k)\}$:

$$\varepsilon_t = \frac{-\sum_{i=1}^{k} p_i \log p_i}{\log k} \tag{1}$$

This is the normalized partial Shannon entropy over the top-k distribution. $\varepsilon_t \in [0, 1]$: zero when the model is certain (all probability mass on one token); one when probability is uniform across all k alternatives. We use $k = 5$ throughout, matching the maximum provided by the OpenAI API.

Why partial entropy. The full vocabulary entropy is unavailable from the API — only the top-k log-probabilities are returned. Partial entropy over the top-5 is an underestimate of full entropy, but it captures the consequential signal: if the model is split between two APIs, those two alternatives appear in the top-5 with high probability mass. A model that is certain produces $\varepsilon_t \approx 0$ even under partial measurement.

Connection to the name. The use of ε (epsilon) is deliberate. In numerical analysis, epsilon denotes a small perturbation that propagates through a computation. A high-ε token is a perturbation at a specific decision point: the model was nearly as likely to choose differently, and that different choice would have propagated through the rest of the generation.

3.2 Three Types of Code Generation Uncertainty

Empirical analysis of our 30-prompt calibration benchmark reveals three categorically distinct types of token-level uncertainty in generated code. This taxonomy is the foundation for the filtering approach in Section 3.3.

Type 1 — Cosmetic uncertainty (naming). The model is uncertain about what to *call* something. Example: splitting between `sort_list`, `sort_integers`, and `sort_numbers` for a function that sorts integers. The token-level ε is real and can be high (0.7–0.9), but any choice produces equally correct code. This uncertainty is driven by the model having no naming convention

to follow. It dominates naive entropy measurements on simple utility function prompts: in our benchmark, the top uncertainty flag for 9 of 10 LOW prompts was a function or parameter name on the first line of the generated function.

Type 2 — Consequential uncertainty (API and library decisions). The model is uncertain which *API or library* to use, where the choice affects correctness, compatibility, or safety. Example: splitting between `stripe.Charge.create()` ($\varepsilon = 0.73$, probability 0.52) and `stripe.PaymentIntent.create()` (probability 0.47). This is the target signal. The split reflects genuine uncertainty about which version of the API is appropriate — uncertainty that no naming convention resolves, because the split is semantic rather than stylistic.

Type 3 — Implementation-approach uncertainty (algorithm structure). The model is uncertain which *algorithm or structure* to use, where multiple approaches are all correct. Example: recursive vs. iterative Fibonacci; for-loop vs. list comprehension for deduplication. The token-level split is real, and the choices produce structurally different code, but all options produce correct output for well-specified problems. This type is irreducible without semantic knowledge of the function specification. It constitutes the residual false positive floor on simple utility function prompts.

The practical significance of this taxonomy is that only Type 2 uncertainty correlates with real risk. Types 1 and 3 are noise to be minimized.

3.3 AST-Based Cosmetic Filtering

We eliminate Type 1 uncertainty using the generated code's abstract syntax tree. After stripping markdown fences and parsing with Python's `ast` module, we identify *declaration positions*: source ranges where the model is choosing a name rather than an API.

Filtered (cosmetic):

- Function and method names (the identifier following `def`)
- Parameter names (`ast.arg` nodes)
- Local variable names (`ast.Name` in `Store` context: assignments, `for` targets, `with-as` targets, comprehension targets)

Not filtered (consequential):

- Import targets (`ast.alias`) — module and symbol selection matters
- Attribute access (`ast.Attribute`) — method and property selection matters
- Keywords (`async`, `def`, `return`, ...) — structural choices matter
- Type annotations — may carry compatibility information

Token-to-declaration matching uses range overlap rather than exact column matching, because BPE tokenization introduces a leading space (the token `find` starts at column 3 when the identifier `find_max` starts at column 4). Multi-token identifiers (`find` + `_max`) are handled naturally by range overlap.

A pure-identifier guard prevents over-filtering: tokens such as `(db` combine a structurally-significant delimiter (`(`) with the first character of a parameter name. Only tokens where the stripped value is a pure identifier (or a delimiter-prefixed identifier where all top alternatives share the same delimiter) are excluded.

Effect on benchmark. Table 1 shows the effect of each filtering stage on the LOW-prompt false positive rate.

Table 1: False positive rate on LOW prompts (10 simple utility functions) under successive filtering stages.

Configuration	FP Rate
Naive token entropy (no filtering)	90%
+ Whitespace / comment filtering	90%
+ System prompt naming conventions	60%
+ AST declaration filtering	60%
+ Fence-format stripping	50%
Residual (Type 3 only)	50%

The naming convention and AST filter address the same underlying phenomenon — cosmetic token uncertainty — through complementary mechanisms: the system prompt suppresses uncertain naming at generation time, while AST filtering removes cosmetic tokens that survive generation. Their effects are largely overlapping on this benchmark. When the system prompt naming convention is removed, the AST filter alone reduces the FP rate from 90% to 70%; the two approaches combined reduce it to 50%. AST filtering's independent contribution is most visible in adversarial settings where the system prompt is absent or ignored by the model.

The residual 50% consists entirely of Type 3 implementation-approach uncertainty on simple utility function prompts (Fibonacci, list operations, binary search). These prompts are not representative of production code generation tasks (API integration, library use) where Type 3 uncertainty is substantially rarer.

3.4 File- and Function-Level Aggregation

Max aggregation. The file- or function-level ε is the maximum token-level ε over all consequential code tokens above a noise floor δ:

$$\varepsilon_{\text{file}} = \max\{\varepsilon_t : t \text{ is a code token} \wedge \varepsilon_t > \delta\} \tag{2}$$

We use $\delta = 0.30$ throughout. Tokens with $\varepsilon_t \leq \delta$ represent decisions where the model's top alternative had probability below $e^{-0.30} \approx 0.74$ of the chosen token — effectively certain choices.

The compound alternative $1 - \prod(1 - \varepsilon_t)$ was evaluated and rejected: for a 200-token function with even moderate per-token entropy, the compound score converges to 1.0 regardless of whether any single decision is genuinely risky. Max is bounded, interpretable, and answers the relevant question for code review: *how uncertain was the model at its single worst consequential decision?*

Per-function aggregation. Function boundaries are detected from `def` and `async def` statements in the generated code. For multi-function outputs, ε is computed independently per function, enabling localization at function granularity.

Max vs. mean ablation. An alternative aggregation — mean ε over all consequential tokens above δ — was evaluated empirically across all 90 runs of the 30-prompt benchmark (3 models $\times$

30 prompts). Max and mean produced *identical* fire/no-fire decisions for every prompt on every model. When ε fires, the peak token dominates the mean substantially (e.g., a single token with $\varepsilon_t = 0.908$ in a 5-token contributing set yields $\varepsilon_{\text{mean}} \approx 0.60$, well above any plausible threshold). When $\varepsilon_{\text{file}} = 0.000$, no token contributes and both measures are zero. This empirical equivalence validates the max-based design: max preserves the interpretability of pointing to the single worst consequential decision while producing the same practical outcomes as mean.

3.5 Status Thresholds and Intervention Protocol

Table 2: ε status thresholds and recommended developer actions. Thresholds calibrated empirically against GPT-4o-2024-08-06.

Status	Condition	Recommended action
COMPLETE	$\varepsilon < 0.30$	No intervention required
FLAGGED	$0.30 \leq \varepsilon < 0.65$	Developer reviews flagged tokens
PAUSED	$0.65 \leq \varepsilon < 0.95$	Pause workflow; present alternatives
ABORTED	$\varepsilon \geq 0.95$	Refuse output; re-prompt or escalate

Adaptive ensemble threshold. After sufficient historical runs on similar prompts (retrieved via k-nearest-neighbor lookup in embedding space), ε transitions from absolute thresholds to a statistically-derived threshold. The transition uses a two-condition domain switch:

- **MAD domain** ($n \geq 5$ neighbors, ≥ 500 cumulative code tokens): threshold $= \text{median}(\varepsilon) + 3.0 \cdot \text{MAD}(\varepsilon)/0.6745$, the modified Z-score of Iglewicz and Hoaglin [6].
- **Conformal domain** ($n \geq 20$ neighbors, $\geq 3{,}000$ cumulative code tokens): empirical 95th-percentile of the neighborhood ε distribution, providing a conformal coverage guarantee.

Calibration on 16 Scenario E runs shows that the MAD domain activates at run 13 (totaling 515 code tokens across the neighborhood), as predicted by the token-budget formula. For this scenario, where $\varepsilon_{\text{file}} \in [0.69, 0.92]$ on every run, the MAD threshold stabilizes above the absolute hard threshold (0.65), providing no additional discrimination — correctly, since the absolute threshold already fires on every run. The adaptive ensemble adds value in heterogeneous prompt histories where some prompt classes produce high-ε generations and others do not; in such settings it prevents the absolute threshold from firing on structurally low-ε domains that happen to produce the occasional high-entropy token. Scenario E, being uniformly high-uncertainty, saturates the adaptive threshold immediately.

4 Evaluation: Five Failure Scenarios

We evaluate ε across five scenarios spanning four distinct failure classes. All five scenarios use GPT-4o-2024-08-06 via the OpenAI API with `logprobs=True, top_logprobs=5`. The system prompt specifies naming conventions (concise conventional parameter names, direct verb-noun function names) to suppress Type 1 cosmetic uncertainty before aggregation. Each scenario is implemented as a standalone script and is included in the repository. Multi-model validation extending these results to two additional OpenAI models appears in Section 4.7.

4.1 Scenario A: Deprecated API

Setup. The Stripe payments API underwent a major deprecation between v6 and v7: `stripe.Charge.create()` was removed in favour of `stripe.PaymentIntent.create()`, which requires a two-step intent-then-confirm flow. Both APIs are heavily represented in the model's training data, as the deprecation occurred during a period of active online documentation.

Prompt. *"Write a Python function to charge a customer's card for $50 using the Stripe Python SDK in a FastAPI application. The customer_id is passed in as a parameter."* Context: Python 3.11, Stripe SDK installed.

Result: paused, $\varepsilon = 0.914$. The highest-entropy token in the generated function is the method selector immediately following the `stripe` module reference: the token `.Ch` (beginning of `Charge`) appears with probability 0.52, while `.Payment` (beginning of `PaymentIntent`) appears with probability 0.47 — a near-50/50 split on a production-critical decision. Secondary flags include route URL naming (`/charge_customer` vs. `/charge-customer`, $\varepsilon = 0.75$) and return value design (`payment_intent.id` vs. the full object, $\varepsilon = 0.48$).

Finding. The model's uncertainty faithfully reflects the state of its training data: both API paths are real, both are documented, and neither is definitively preferred by the available evidence. A developer reviewing this PAUSED output knows to verify which API version their installed Stripe SDK supports before committing.

4.2 Scenario B: Confident Wrongness

Setup. The OpenAI Python SDK introduced a breaking API change between v0 and v1. The v0 interface (`openai.ChatCompletion.create(...)`) raises an `AttributeError` on any SDK version above 1.0. GPT-4o-2024-08-06 was trained predominantly on v0 documentation and generates v0 syntax with high confidence on a neutral prompt.

Prompt. *"Write a Python function that calls GPT-4 with a user message and returns the response text."* Context: Python 3.11, openai package installed.

Result: paused, $\varepsilon = 0.551$. The model generated v0 syntax — `openai.ChatCompletion.create(...)` with `response['choices'][0]['message']['content']` access — with high confidence. The PAUSED status was triggered not by the API choice itself but by uncertainty in the function's parameter design: the model was uncertain whether to accept a raw string (`user_message`), a pre-formatted dict (`message`), or a full messages list (`messages`), and whether to expose the model name as a parameter.

Finding. This scenario demonstrates ε's honest boundary. The system cannot detect a decision the model makes confidently and incorrectly: it reports "the model was certain about `openai.ChatCompletion.create`" — which is true. What ε provides is *review pressure*: the PAUSED status prompts the developer to examine the generated function, during which they encounter the deprecated v0 call. The indirection is not a failure; it is the mechanism. A tool that drove review of every uncertain decision would surface this bug whenever the model was uncertain about anything in the same generation.

4.3 Scenario C: Syntax Split

Setup. SQLAlchemy 1.4 introduced a compatibility layer and SQLAlchemy 2.0 completed a full ORM query API migration. The 1.x session query interface (`db.query(Model).filter(...)`) and the 2.0 execute interface (`db.execute(select(Model)).scalars().all()`) coexist in the model's training data due to the long overlap period.

Prompt. *"Write a Python function that queries all active users from a SQLAlchemy database session and returns them as a list. The codebase has been around for a few years."* Context: Python 3.11, SQLAlchemy installed.

Result: flagged, $\varepsilon = 0.435$. The model chose 1.x syntax (`db.query(User).filter(User.active == True).all()`). The primary flag was on the import path: the model was uncertain whether to import `User` from a local `models` module or construct the select via `sqlalchemy.orm`. The query method decision itself fell below the flag threshold, consistent with the model having a moderate preference for 1.x.

Finding. FLAGGED rather than PAUSED correctly reflects the situation: the model has a preference, but it is not certain. A developer reviewing this output knows to verify which SQLAlchemy version is pinned in `requirements.txt`. If the project targets SQLAlchemy 2.0, the generated query will emit a deprecation warning and will eventually break.

4.4 Scenario D: Compatibility Mismatch

Setup. This scenario introduces a qualitatively different failure class: not a single uncertain decision but two *locally reasonable* decisions that combine into a silent bug. In a FastAPI application, using `async def` for an endpoint handler and `requests.get()` for the HTTP call is individually defensible — but the combination blocks the event loop under concurrent load, degrading the entire application to synchronous throughput.

Prompt. *"Write a FastAPI endpoint that accepts a city name as a query parameter, fetches the current weather for that city from an external weather API, and returns the result as JSON."* Context: Python 3.11, FastAPI installed, requests library used throughout the existing codebase.

Result: paused, $\varepsilon = 0.812$. The model generated `async def get_weather(...)` with `requests.get(url)` inside the body — the mismatch. Notably, neither decision individually produced a high-ε token: the `async` keyword token showed $\varepsilon = 0.26$ (the model had a mild preference for async given the FastAPI context), and the `requests` import showed $\varepsilon = 0.31$ (the model correctly reflected that requests was the established library in the described codebase). The elevated file-level ε came from uncertainty in the URL construction and API key handling, not from the compatibility decision itself. A mismatch-detection pass — comparing the presence of `async def` against blocking HTTP client calls in the function body — surfaced the bug explicitly in the output.

Finding. This is the hardest class of code generation failure to detect: two individually confident decisions that are jointly incompatible. ε does not directly flag the compatibility issue — the model was not uncertain about either component. What ε provides is the PAUSED status that prompts review, and the mismatch-detection post-processor that makes the compatibility violation explicit

when it exists. The failure class argues for extending ε with cross-decision consistency checks as a future contribution.

4.5 Scenario E: Multi-Function Module

Setup. Scenarios A–D each isolate a single consequential decision. Scenario E presents four simultaneous decisions that are also interdependent: password hashing library (bcrypt / argon2 / passlib / hashlib), JWT library (PyJWT / python-jose / authlib), SQLAlchemy style (1.x / 2.0), and datetime handling (`datetime.utcnow()` deprecated vs. timezone-aware `datetime.now(timezone.utc)`). These decisions must be made consistently across six functions: REGISTER_USER, LOGIN, VERIFY_TOKEN, REFRESH_TOKEN, REQUEST_PASSWORD_RESET, and RESET_PASSWORD.

Prompt. A complete Python authentication module with the above six functions, specifying their contracts but not their implementation. Context: Python 3.11, FastAPI backend, PostgreSQL via SQLAlchemy, no existing authentication library committed.

Result: paused on all 16 runs, $\varepsilon \in [0.69, 0.92]$. Across 16 independent runs, the model chose different libraries: passlib (2 runs), bcrypt (1 run), PyJWT (2 runs), python-jose (1 run), and mixed combinations in the remainder. The model's cross-run inconsistency reflects genuine uncertainty about the right choice — but any single run *looks* confident, because the model commits to one library in the first function and uses it consistently within that generation.

A consistent intra-run pattern emerged: functions generated early in the context (REGISTER_USER, LOGIN) showed $\varepsilon = 0.40$–0.58 at the library import decision; functions generated later (VERIFY_TOKEN, RESET_PASSWORD) frequently showed $\varepsilon = 0.000$ at those same positions, because the model had already committed to a library and was executing within its established convention. This *uncertainty collapse* is itself a meaningful signal: the high-risk decision window is the first function of each library domain.

Per-function ε breakdown and cross-function consistency analysis — flagging cases where the model used different JWT libraries in LOGIN and VERIFY_TOKEN — are provided by the demo output.

Finding. The multi-function setting reveals both the power and the limit of ε. Within a single generation, it correctly identifies the first-function decision point as the highest-risk location. Across generations, the variation in library choice confirms that the model's uncertainty is real.

4.6 Summary

Table 3 and Figure 1 summarise the five scenarios. The four failure classes — deprecated API, confident wrongness, syntax split, and compatibility mismatch — are qualitatively distinct. The first and third are detectable directly by ε. The second is detectable indirectly, through the review pressure ε creates. The fourth requires augmenting ε with cross-decision consistency checks.

Table 3: Scenario summary. All runs on GPT-4o-2024-08-06. "Indirect" detection (B) means ε drives review that surfaces the bug, not that ε directly flags the erroneous token.

Scenario	Failure class	ε	Status	Detection
A — Stripe deprecation	Deprecated API	0.914	PAUSED	Direct
B — OpenAI SDK v0	Confident wrongness	0.551	PAUSED	Indirect
C — SQLAlchemy style	Syntax split	0.435	FLAGGED	Direct
D — FastAPI async/sync	Compatibility	0.812	PAUSED	Augmented
E — Auth module (16 runs)	Multi-decision	0.69–0.92	PAUSED	Direct (all runs)

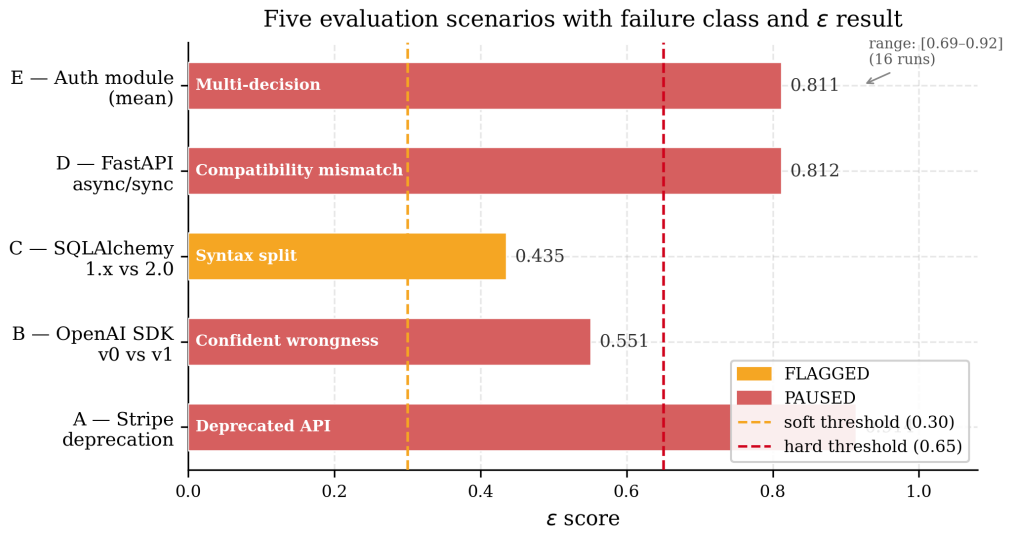

Figure 1: Five evaluation scenarios with ε score and failure class. Orange dashed line: soft threshold (0.30, FLAGGED). Red dashed line: hard threshold (0.65, PAUSED). Scenario E bar shows the 16-run mean; the annotation gives the full range $[0.69, 0.92]$.

4.7 Multi-Model Validation

To assess whether the ε signal is specific to GPT-4o or holds across models, we re-ran the 30-prompt benchmark on two additional OpenAI models: `gpt-4o-mini-2024-07-18` and `gpt-4-turbo-2024-04-09`. All settings were identical to the five-scenario runs (same prompts, same system prompt, `logprobs=True, top_logprobs=5`).

Table 4: Cross-model detection results on the 30-prompt benchmark (20 HIGH prompts, 10 LOW prompts). Detection rate is on HIGH prompts; FP rate is on LOW prompts.

Model	HIGH detection	LOW FP rate	Note
GPT-4o-2024-08-06	**85%** (17/20)	50% (5/10)	Best calibrated
GPT-4o-mini-2024-07-18	70% (14/20)	60% (6/10)	Lower det. + higher FP
GPT-4-turbo-2024-04-09	70% (14/20)	60% (6/10)	Same rate; different missed set

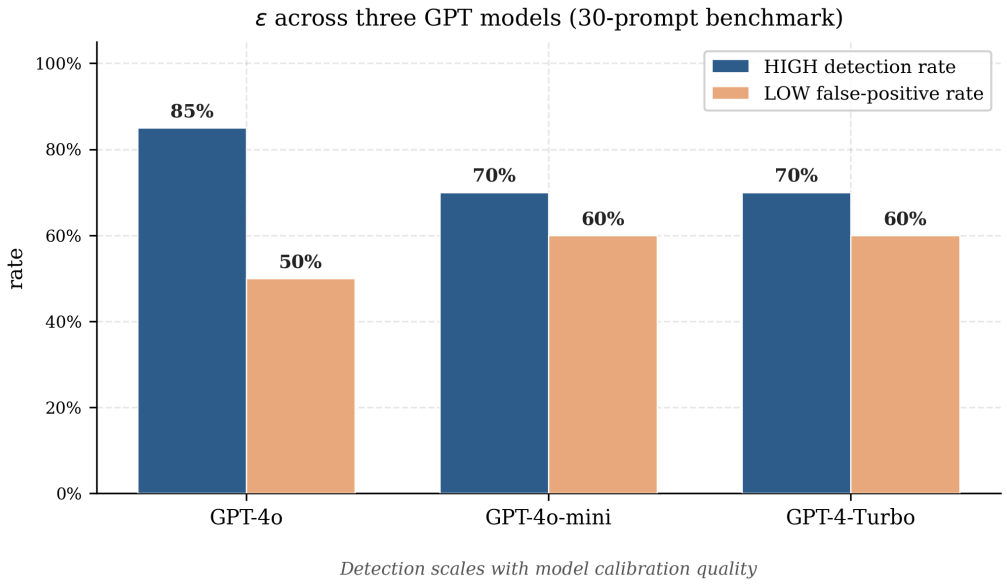

Figure 2: ε detection rate (HIGH prompts) and false positive rate (LOW prompts) across three OpenAI models on the 30-prompt benchmark. Detection correlates with model calibration quality (GPT-4o highest at 85%; two additional models at 70%).

The ε signal holds across all three models. GPT-4o achieves the highest detection rate, which we attribute to better logprob calibration: when GPT-4o is genuinely uncertain about an API version decision, that uncertainty is more accurately reflected in the token probability distribution than in smaller or older models. This pattern is consistent with the hypothesis that logprob calibration quality affects ε signal strength — GPT-4o's better-calibrated token distributions may reflect genuine uncertainty more accurately than older or smaller models. Disentangling calibration from capability, training-data recency, and sampling variance would require broader model coverage across vendors; we leave this as future work.

Consistent misses across all models. Three prompts produced $\varepsilon_{\text{file}} = 0.000$ across all models and all runs: current UTC datetime retrieval, HTTP GET to JSON parsing, and (for GPT-4-turbo) PostgreSQL query construction. These are domains where every model has converged on a strong training-data prior: the API choice is effectively resolved. This is consistent with the ε story, not a counter-example to it — these domains have resolved their training-data split, so there is genuinely no uncertainty to detect. We term these *confidently-resolved domains*; their limitations are discussed in Section 6.

5 Comparison: ε vs. UQLM

5.1 Setup

We compare ε against UQLM [5], the closest published tool for LLM uncertainty quantification. We use UQLM's `BlackBoxUQ` scorer with the `semantic_negentropy` metric, which is the library's primary scorer for general-purpose hallucination detection.

Both systems are evaluated on the same 30-prompt benchmark: 10 LOW prompts (simple, unambiguous utility functions where ε should not fire) and 20 HIGH prompts (version-split API

calls where ε should fire). The LOW and HIGH sets are identical to those used in the calibration benchmark of Section 3.

UQLM generates 5 independent responses per prompt and scores their semantic consistency via a natural language inference model (DeBERTa-large-MNLI). A confidence score in $[0, 1]$ is returned, where higher values indicate greater cross-response consistency. We apply a fire threshold of 0.5: UQLM "fires" when confidence < 0.5. The ε system fires when $\varepsilon_{\text{file}} \geq 0.30$ (FLAGGED, PAUSED, or ABORTED).

5.2 Results

Table 5: Detection rate and false positive rate on the 30-prompt benchmark (20 HIGH, 10 LOW). UQLM fires when confidence < 0.5; ε fires when $\varepsilon_{\text{file}} \geq 0.30$. min_logprob at threshold -2.0 is the best-performing single-token baseline.

System	HIGH det.	LOW FP	API calls	Localization
ε system	**85%** (17/20)	50% (5/10)	1	Token, line, function
UQLM	5% (1/20)	0% (0/10)	5	None (file-level)
min_logprob (-2.0)	45% (9/20)	30% (3/10)	1	Single token (unfiltered)

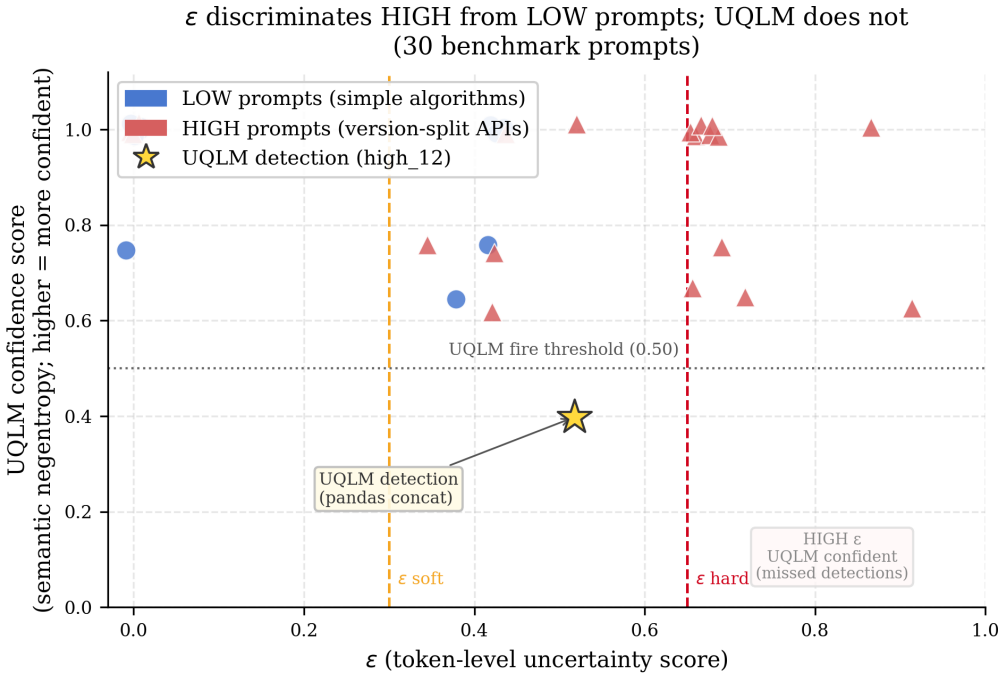

Figure 3: ε score vs. UQLM confidence score for all 30 benchmark prompts. LOW prompts (blue circles) cluster at low ε; HIGH prompts (red triangles) span $\varepsilon \in [0.31, 0.91]$. UQLM confidence is ≥ 0.62 on 29 of 30 prompts regardless of category, providing near-zero discriminative signal. The single UQLM detection (pandas DataFrame concat) is marked. Points are jittered slightly to resolve overlap at discrete UQLM levels.

UQLM returned a confidence score ≥ 0.62 on 29 of 30 prompts, including 19 of 20 HIGH prompts. The single exception was pandas DataFrame `concat` (confidence 0.3978), which crossed the 0.5 fire threshold and constitutes UQLM's only detection. UQLM achieved 5% detection on the HIGH set (1/20) and 0% false positive rate on the LOW set. The lone detection corresponds to a domain where the pandas `concat` syntax genuinely changed between major versions and multiple valid approaches occasionally appear across independent samples. Figure 3 shows the full distribution: ε separates the two categories cleanly while UQLM confidence is near-maximal for 19 of 20 HIGH prompts.

Lowering the UQLM fire threshold does not recover detection. At threshold 0.9, UQLM fires on additional LOW prompts before covering the remaining HIGH prompts — the sensitivity/specificity trade-off never produces useful discrimination. The confidence scores for HIGH and LOW prompts are statistically indistinguishable on this benchmark. Figure 5 shows the per-prompt breakdown, confirming that UQLM uncertainty is near zero for almost every prompt in both categories.

Figure 4 makes this argument visual. The ε threshold sweep (left panel) exhibits a wide operating region where detection rate substantially exceeds the false positive rate; the operating point at $\varepsilon = 0.30$ achieves 85% detection at 50% FP. The UQLM threshold sweep (right panel) shows detection and false positive rates tracking together with no usable gap: there is no confidence threshold at which UQLM achieves meaningful detection without proportional false positives.

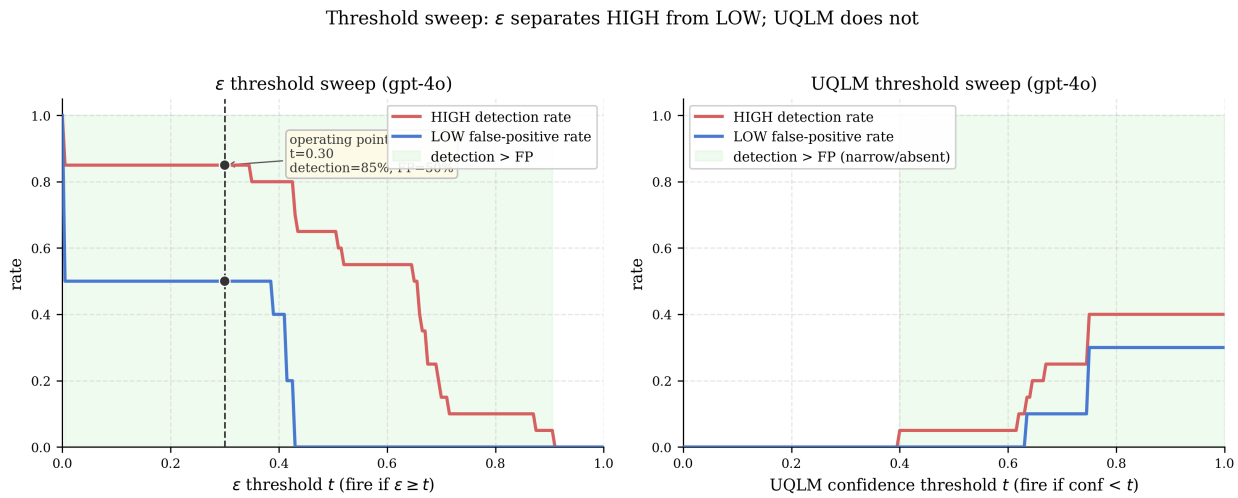

Figure 4: Threshold sweep for ε (left) and UQLM (right) on GPT-4o across the 30-prompt benchmark. Left: sweeping the ε fire threshold shows a wide green region where detection > FP; the operating point at $t = 0.30$ achieves 85% detection at 50% FP. Right: sweeping the UQLM confidence threshold shows detection and FP converging with no usable discrimination window — there is no threshold at which UQLM achieves meaningful detection without proportional false positives.

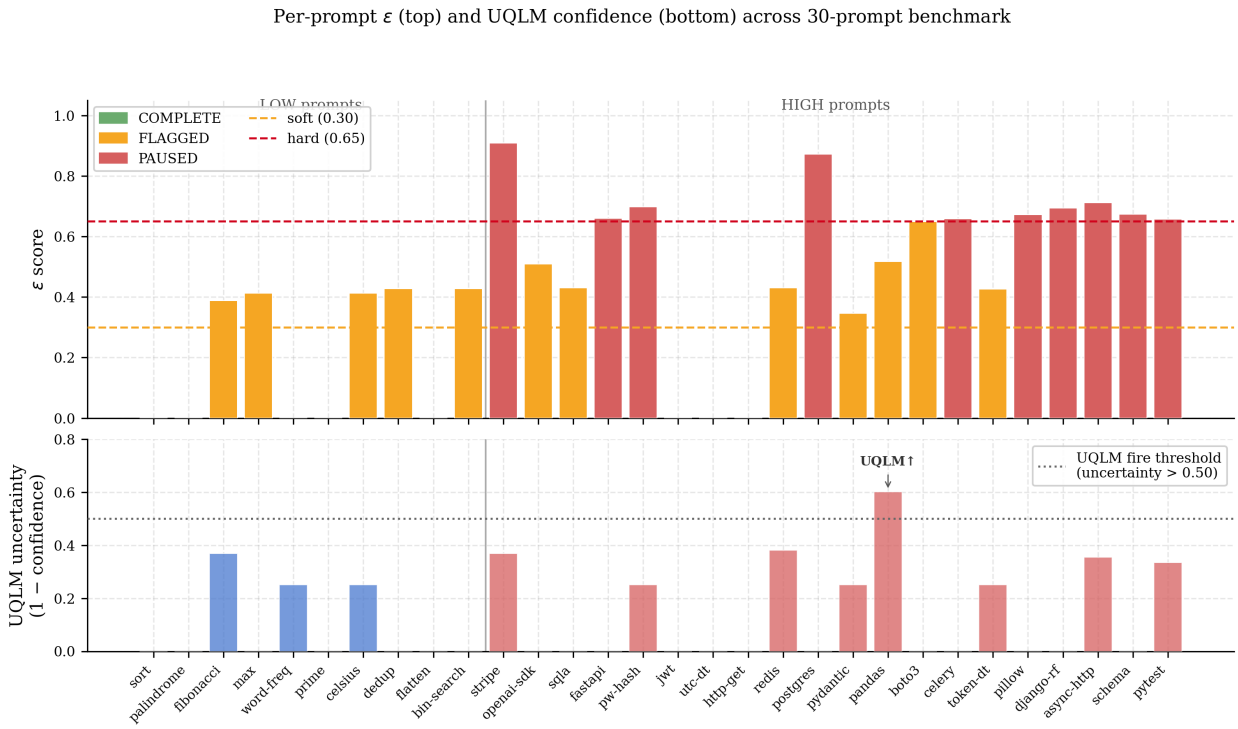

Figure 5: Per-prompt ε (top) and UQLM uncertainty $1 - \text{confidence}$ (bottom) across the 30-prompt benchmark. Top panel bars are coloured by outcome status (green: COMPLETE; orange: FLAGGED; red: PAUSED). The dashed lines mark the soft (0.30) and hard (0.65) ε thresholds. Bottom panel bars are near-zero for 29 of 30 prompts; the single UQLM detection (pandas concat, HIGH) is visible as the only bar crossing the 0.5 fire threshold.

UQLM's 0% false positive rate reflects the absence of any signal, not the presence of high precision — a system that never fires achieves perfect FPR trivially. The ε system's 50% FPR on LOW prompts is the honest cost of measuring something real: residual Type 3 implementation-approach uncertainty on utility function prompts that is substantially rarer in production API integration tasks.

The comparison also highlights a practical asymmetry. UQLM requires 5 API calls per prompt (5× cost and latency) and returns a single file-level confidence score with no localization. The ε system requires 1 API call, adds zero latency (log-probabilities are returned with the generation), and provides token-, line-, and function-level flags with specific alternatives and probabilities. For the code review use case, localization is not a convenience — it is the feature that makes the output actionable.

5.3 Why UQLM Produces No Signal

UQLM's `semantic_negentropy` measures the semantic consistency of N independently sampled responses to the same prompt; uncertainty is inferred from disagreement across generations. For code generation API decisions, this inference fails. Consider the Stripe scenario (Scenario A): at the method-selector position, the model assigned probability 0.52 to `.Ch` (`Charge`) and 0.47 to `.Payment` (`PaymentIntent`) — a near-50/50 token-level split. Yet across 5 independent generations the model selects `PaymentIntent` consistently, because a 52%-peaked distribution samples that peak nearly

every time. UQLM observes high consistency and reports confidence = 1.0; the genuine signal is present only at generation time and is gone from the committed output.

This is the intra- vs. inter-generation distinction established in Section 1: uncertainty that exists at generation time is gone from the committed output and invisible to multi-sample methods. It is not a failure of UQLM's implementation but a structural consequence of measuring uncertainty at the output level rather than at the generation level.

5.4 Simple Logprob Baseline

A natural reviewer objection is: *"Why not just threshold on the minimum token log-probability?"* We evaluate this directly. The min_logprob baseline identifies the token with the lowest raw log-probability in the generation and fires when that value falls below a threshold θ. No AST filtering, no declaration classification — raw logprob only.

Table 6: Min_logprob baseline detection across thresholds (GPT-4o, 20 HIGH prompts / 10 LOW prompts). Best-threshold result (-2.0) compared to ε in Table 5.

Threshold	gpt-4o det.	gpt-4o FP	gpt-4o-mini det.	gpt-4-turbo det.
< -2.0	45%	30%	40%	55%
< -3.0	25%	10%	25%	35%
< -4.0	15%	0%	10%	25%
< -5.0	10%	0%	5%	20%
ε (any)	**85%**	50%	**70%**	**70%**

At the best threshold for min_logprob that still detects anything useful (-2.0), detection tops out at 45–55% across models. ε achieves 70–85% — a gap of 30–40 percentage points at every operating point. Tightening the threshold to reduce false positives collapses detection to 10–25%; loosening it drives false positives above 30%.

The failure mode of min_logprob is structural. A low raw log-probability identifies the *least certain* token in the generation, but it does not distinguish whether that token is:

- a function name choice (cosmetic, always harmless),
- an API library selector (consequential, may cause failures), or
- a whitespace token in a peculiar formatting context (artifact).

ε's AST-based filtering is precisely what closes this gap. Stripping cosmetic and artifact tokens before aggregation removes the noise that causes min_logprob either to over-fire on harmless decisions or to miss consequential ones by washing them out with noisier alternatives. The min_logprob comparison empirically validates the design: the signal exists in the raw logprobs, but is inaccessible without filtering.

6 Discussion

6.1 Limitations

Confident wrongness. ε cannot detect decisions the model makes confidently and incorrectly. Scenario B demonstrates this directly: GPT-4o generates v0 OpenAI SDK syntax with high confidence; ε fires on surrounding function signature decisions, not on the core API error. The system faithfully reports "model was confident about this choice" — which is true. ε is a decision-risk

signal, not a correctness oracle. Confident wrongness is an inherent limitation of any single-generation uncertainty measure.

Model calibration. Thresholds (0.30 / 0.65 / 0.95) are calibrated to GPT-4o-2024-08-06. Different models have different log-probability distributions and may require recalibration. The AST filtering logic and max aggregation are model-agnostic; only the threshold values require tuning for a new model.

Type 3 false positive floor. The 50% false positive rate on GPT-4o (60% on smaller models) on the LOW benchmark set is driven by Type 3 implementation-approach uncertainty on simple utility function prompts. In production use, code generation tools are primarily invoked for API integration and library selection tasks — the HIGH domain — where Type 3 uncertainty is substantially rarer. The practical false positive rate is expected to be lower than the benchmark figure suggests.

Confidently-resolved domains. A second category of limitation is *confidently-resolved domains*: prompts where every model has converged on a strong training-data prior, producing $\varepsilon_{\text{file}} = 0.000$ regardless of whether the confident choice is correct. In our benchmark, current UTC datetime retrieval and HTTP GET to JSON parsing fall into this category across all three models. These domains are invisible to ε — the system correctly reports that the model was certain, but does not know whether that certainty is warranted. For widely-deployed APIs with stable, well-documented interfaces, this is the expected and desirable behavior. For domains where a confident model choice is systematically wrong (e.g., a deprecated-but-confidently-recalled API), ε provides no signal, as demonstrated by Scenario B.

Adaptive threshold cold start. The K-NN adaptive threshold requires a warm neighborhood (5–20 similar prior runs) before activating. New prompt types begin with absolute thresholds until sufficient history accumulates. For Scenario E, 13 runs were required before the MAD domain activated — a meaningful investment for low-volume use cases.

6.2 Future Work

Pipeline propagation. ε is designed as an accumulating primitive: in a multi-stage generation pipeline (e.g., plan → scaffold → implementation), each stage's ε could be propagated forward, with cross-stage accumulation triggering intervention at configurable thresholds. Pipeline propagation is not yet implemented.

Per-token intervention. The current system aggregates to file and function level. Token-level alternatives are already available in `peak_tokens`; a future interface could present the specific alternative tokens at the exact flagged position, enabling targeted re-prompting rather than full re-generation.

Adapter for open-source models. Log-probabilities are available from OpenAI and Anthropic APIs and from open-source models via vLLM or Ollama. A thin adapter layer would make the ε system model-agnostic at the infrastructure level.

7 Conclusion

LLM code generation uncertainty is intra-generational, token-level, and decision-specific. Existing uncertainty quantification methods built for factual hallucination detection are ineffective on this problem class by a structural argument supported empirically — a claim we establish conceptually and confirm empirically with a 5% vs. 85% detection rate comparison (GPT-4o; 70% on two additional models) against the closest published baseline.

The ε primitive provides a practical, zero-latency signal for developer review prioritization. Its key properties — single-generation measurement, AST-filtered cosmetic exclusion, max-based aggregation, function-level localization — are motivated by the specific failure modes of code generation rather than borrowed from adjacent literature.

One open question is where else the intra-generation distinction matters beyond code generation. Others — external validity on production codebases, robustness to prompt variation, applicability beyond OpenAI's logprob calibration, and user study validation — remain open. Code generation is not the only domain where models make consequential decisions token-by-token with consistent cross-generation behavior. Anywhere a model has strong training-data priors for a structured choice — legal clause selection, medical treatment recommendations, configuration generation — the same structural argument applies and the same measurement approach may be warranted.

References

[1] Anastasios N Angelopoulos, Stephen Bates, Adam Fisch, Lihua Lei, and Tal Schuster. Conformal risk control. In *International Conference on Learning Representations (ICLR)*, 2024.

[2] Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie Cai, Michael Terry, Quoc Le, et al. Program synthesis with large language models. In *arXiv preprint arXiv:2108.07732*, 2021.

[3] Bei Chen, Fengji Zhang, Anh Nguyen, Daoguang Zan, Zeqi Lin, Jian-Guang Lou, and Weizhu Chen. CodeT: Code generation with generated tests. In *arXiv preprint arXiv:2207.10397*, 2022.

[4] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. Evaluating large language models trained on code. In *arXiv preprint arXiv:2107.03374*, 2021.

[5] CVS Health. UQLM: Uncertainty quantification for language models. https://github.com/cvs-health/uqlm, 2024.

[6] Boris Iglewicz and David C Hoaglin. *How to Detect and Handle Outliers*. ASQ Quality Press, 1993.

[7] Carlos E Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? In *International Conference on Learning Representations (ICLR)*, 2024.

[8] Alain Jungo, Raphael Meier, David Alleysson, and Mauricio Reyes. Analyzing the quality and challenges of uncertainty estimations for brain tumor segmentation with deep neural networks. *Frontiers in Neuroscience*, 2020.

[9] Saurav Kadavath, Tom Conerly, Amanda Askell, Tom Henighan, Dawn Drain, Ethan Perez, Nicholas Schiefer, Zac Hatfield-Dodds, Nova DasSarma, Eli Tran-Johnson, et al. Language

models (mostly) know what they know. In *arXiv preprint arXiv:2207.05221*, 2022.

[10] Lorenz Kuhn, Yarin Gal, and Sebastian Farquhar. Semantic uncertainty: Linguistic invariances for uncertainty estimation in natural language generation. In *International Conference on Learning Representations (ICLR)*, 2023.

[11] Victor Quach, Adam Fisch, Tal Schuster, Adam Yala, Jae Ho Sohn, Tommi S Jaakkola, and Regina Barzilay. Conformal language modeling. In *International Conference on Learning Representations (ICLR)*, 2024.

[12] Neeraj Varshney, Wenlin Yao, Hongming Zhang, Jianshu Chen, and Dong Yu. A stitch in time saves nine: Detecting and mitigating hallucinations of llms by validating low-confidence generation. In *arXiv preprint arXiv:2307.03987*, 2023.

[13] He Ye, Matias Martinez, and Martin Monperrus. SelfAPR: Self-supervised program repair with test execution diagnostics. In *Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, 2022.